



PEC 1 - Representación de problemas

Presentación

Primera PEC del curso de Inteligencia Artificial.

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.

Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre formalización de problemas y búsqueda básica sobre espacio de estados.

Descripción de la PEC

Consultad la sección 1.1.1 del módulo de “Resolución de problemas y búsqueda” de los materiales de la asignatura. Ahí, en la página 15 (al final) y en la 17, se plantea el problema de las ocho reinas, así como una posible modelización. En esta PEC, vamos a partir de esta modelización.

Responded a las siguientes cuestiones:

1. (1 punto) ¿Cuántos estados posibles tiene la representación planteada? ¿Son todos válidos? Argumentad vuestra respuesta.

8 variables, 9 posibles valores cada una (8 columnas+posición desasignada).

$9 \times 9 \times 9 \times 9 \times 9 \times 9 \times 9 \times 9 = 9^8 = 43046721$ posibles estados.

No todos son válidos, por las propias restricciones del problema.



2. (1 punto) ¿Cuáles son los operadores de los que disponemos? ¿Cómo cambian los estados? ¿Son todos los estados accesibles desde cualquier otro estado? Argumentad vuestra respuesta.

La única acción que se define en la modelización propuesta es “asignar un valor a una variable que todavía no tenía un valor asignado”. Así pues, contamos con un único operador que realizará esta acción.

NO todos los estados serán accesibles desde cualquier otro estado, dado que la acción modelada no permite asignar un valor a una variable que ya tiene valor asignado.

Aparte de este detalle, y dependiendo de la definición del alumno, se puede considerar que el propio operador tiene en cuenta por sí mismo si el estado que va a producir es válido o no.

Si lo tiene en cuenta, y en consecuencia nuestro operador no puede generar estados inválidos, entonces el número de estados accesibles desde cualquier otro estado es mucho menor que en el caso que el operador pueda producir estados inválidos.

3. (5 puntos)

En la sección 2.1 del mismo módulo de los materiales, podemos encontrar el pseudocódigo de la función *busqueda()*. Realizad una definición completa y en pseudocódigo de los siguientes elementos que podemos encontrar en él:

- a) (1.25 puntos) Definid *problema* para ajustarlo al problema de las ocho reinas.

Evidentemente, en función del planteamiento del alumno, un apartado puede ser más extenso que otro, ya que reutilizará funciones uno de otro, por ello todos los apartados tienen la misma puntuación, y esta solución no es la única posible.

Esencialmente, y de acuerdo a la modelización propuesta, el *problema* será una estructura/clase/objeto/entidad que dispondrá de 8 variables para definir su estado, una función para definir el estado inicial, una función para comprobar si el estado es solución, y una función operador, de acuerdo al operador que se haya definido. También puede contener una función para comprobar si un estado es inválido, en caso que el operador no lo compruebe por sí mismo, y una función para inicializar al estado inicial que se indica en la modelización:



```
Clase problema{
    int x[8]; //agrupamos las variables en un array
    fun estado_inicial(){
        for (int i=0; i<8; i++)
            x[i]=NULL;
    }
    fun operador(i, valor){
        x[i] = valor;
    }
    fun check_solucion() -> bool{
        //ha de ser válido y que todas las variables tengan valor
        for (int i=0; i<8; i++)
            if ( x[i] == NULL )
                return false;
        return check_estado_valido();
    }
    fun check_estado_valido() -> bool{
        //que dos reinas no estén en la misma columna
        for (int i=0; i<8; i++)
            for (int j=0; j<8; j++)
                if (i != j && x[i]!=NULL && x[j]!=NULL)
                    if (x[i]==x[j])
                        return false;

        //que dos reinas no estén en la misma diagonal
        for (int i=0; i<8; i++)
            for (int j=0; j<8; j++)
                if ( j > i && x[i]!=NULL && x[j]!=NULL ){
                    if ( (x[i] - x[j]) == (j - i) )
                        return false;
                    if ( (x[j] - x[i]) == (j - i) )
                        return false;
                }
        return true; //si llegamos hasta aquí, estado válido
    }
}
```

- b) (1.25 puntos) Definid la función *solucion*, que recibe como argumentos un problema y un nodo.

```
//considerando el nodo como un vector de 8 posiciones
fun solucion(problema,nodo)->bool {
    problema.x = nodo;
    return problema.check_solucion();
}
```



c) (1.25 puntos) Definid la función *expandir*.

```
//considerando el nodo como un vector de 8 posiciones
fun expandir(problema,nodo)->int[] {
    int nuevos_nodos[][8];
    int nuevo_nodo[8];
    int z=0; //índice del array que se devuelve

    for (int i=0; i<8; i++)
        for (int j=0; j<8; j++){
            /*en cada iteración, reiniciamos estado para
            poder operar, operamos, y si el estado
            producido es válido, lo añadimos a la lista
            a devolver*/

            problema.x = nodo;
            problema.operador(i,j);
            nuevo_nodo = problema.x;
            if(check_estado_valido(nuevo_nodo)){
                nuevos_nodos[z]=nuevo_nodo;
                z++;
            }
        }
    return nuevos_nodos;
}
```

d) (1.25 puntos) Definid *anadir-nodos-a-por-expandir*, con una estrategia de búsqueda fija. La estrategia de búsqueda será siempre la misma, no la recibirá como argumento, es decir, debéis implementarla dentro de esta propia función. ¿Qué estrategia implementáis? ¿Por qué?

Dadas las características de este problema, no parece haber una opción predominante sobre otra ya que, por un lado, la solución se encontrará siempre realizando 8 acciones, independientemente de si se busca en anchura o profundidad, y por el otro, siempre que se compruebe que los nuevos estados generados son válidos antes de expandir (si no son válidos, ni se generan), no se trata de un problema que vaya a comprometer la memoria disponible. A continuación, se define para ambas estrategias:

```
//En profundidad: se añaden los nuevos nodos delante de los que
//ya teníamos pendientes de expandir.
fun anadir-nodos-a-por-expandir-prof(nuevos-nodos,arbol) {
    int nodos_por_expandir[][8];
    nodos_por_expandir = nuevos_nodos+arbol.nodos_a_expandir;
    arbol.nodos_a_expandir = nodos_por_expandir;
}
```



```
//En anchura: se añaden los nuevos nodos detrás de los que
//ya teníamos pendientes de expandir.
fun anadir-nodos-a-por-expandir-anchura(nuevos-nodos, arbol) {
    int nodos_por_expandir[][8];
    nodos_por_expandir = arbol.nodos_a_expandir+nuevos_nodos;
    arbol.nodos_a_expandir = nodos_por_expandir;
}

//Nota: consideramos que el operador + aplicado a listas une dos
//listas en el orden de los operandos
```

Para ello, contad con que ya disponéis de las estructuras *nodo* y *arbol de busqueda*, definidas en los materiales justo antes del pseudocódigo de la función *busqueda()*. Considerad que siempre partimos de un estado inicial donde solo una de las reinas está colocada al azar en una casilla del tablero.

4. (1.5 puntos)

Resolved el problema a partir del estado que se expone a continuación, representando la evolución de la lista de nodos por expandir durante la búsqueda y qué nodo se expande en cada paso, hasta llegar a una solución.

El estado inicial es un tablero con 5 reinas, colocadas en las siguientes posiciones:

1ª fila, 6ª columna

2ª fila, 2ª columna

3ª fila, 7ª columna

4ª fila, 1ª columna

5ª fila, 3ª columna

Nota: Para este ejercicio no es necesario haber realizado el ejercicio 3, aunque es aconsejable.

Importante: la función *expandir* arriba definida no devuelve estados inválidos. Consideramos búsqueda en profundidad.

Paso 0

Estado inicial: $x = [5, 1, 6, 0, 2, \text{NULL}, \text{NULL}, \text{NULL}]$

Nodos por expandir (solo el estado inicial):

[[5, 1, 6, 0, 2, NULL, NULL, NULL]]



Paso 1

Seleccionamos primer nodo de la lista por expandir.

Comprobamos si tal nodo es solución. No lo es. Lo expandimos.

Nodos por expandir:

```
[ [5,1,6,0,2,4,NULL,NULL], [5,1,6,0,2,7,NULL,NULL],  
[5,1,6,0,2,NULL,7,NULL], [5,1,6,0,2,NULL,NULL,3] ]
```

Paso 2

Seleccionamos primer nodo de la lista por expandir.

Comprobamos si tal nodo es solución. No lo es. Lo expandimos.

Añadiremos los nuevos nodos (en negrita) por delante de los anteriores.

Nodos por expandir:

```
[ [5,1,6,0,2,4,7,NULL], [5,1,6,0,2,4,NULL,3],  
[5,1,6,0,2,7,NULL,NULL], [5,1,6,0,2,NULL,7,NULL],  
[5,1,6,0,2,NULL,NULL,3] ]
```

Paso 3

Seleccionamos primer nodo de la lista por expandir.

Comprobamos si tal nodo es solución. No lo es. Lo expandimos.

Añadiremos los nuevos nodos (en negrita) por delante de los anteriores.

Nodos por expandir:

```
[ [5,1,6,0,2,4,7,3], [5,1,6,0,2,4,NULL,3],  
[5,1,6,0,2,7,NULL,NULL], [5,1,6,0,2,NULL,7,NULL],  
[5,1,6,0,2,NULL,NULL,3] ]
```

Paso 4

Seleccionamos primer nodo de la lista por expandir.

Comprobamos si tal nodo es solución.

Efectivamente, lo es.

¡Problema resuelto! Solución: [5,1,6,0,2,4,7,3]



5. (1.5 puntos)

¿Qué cambios supondría, en todos los aspectos de la modelización del problema, si para este problema añadimos una variante que permite poner 2 obstáculos en el tablero que las reinas no pueden atravesar? Es decir, imaginad que existen dos figuras mágicas que ocupan cada una una posición del tablero, y que las reinas no pueden atravesar en sus recorridos.

Nos referimos a cambios sobre la entera modelización inicial, no respecto a lo implementado en el ejercicio 3.

Obviamente, la representación se verá afectada por estos nuevos ítems, para lo que necesitaremos poder representar 4 valores nuevos: la fila y columna de cada uno de ellos dos.

La definición del estado inicial deberá contar la posición de estas piezas.

El operador deberá tener en cuenta la posición de las piezas, también (no podrá colocar una reina en esa posición).

Finalmente, muy importante, las condiciones del problema cambian, ya que a la hora de comprobar si un estado es válido o no (i.e. si una reina mata a otra reina en un determinado estado), estas nuevas piezas juegan un papel fundamental, ya que ahora las posibles soluciones aumentan (por ejemplo, dos reinas pueden estar en una misma columna, si tienen un obstáculo entre ellas). Esto afecta a las comprobaciones de estado válido y estado solución.

Yendo un poco más lejos, incluso, hasta la definición del estado cambiaría, ya que ahora 2 reinas pueden estar en una misma fila, por lo que, en vez de considerar 8 variables donde cada una de ellas representa una fila (reina y fila iban asociadas hasta ahora), ahora estas 8 variables representarán la posición de cada reina en el tablero, por lo que tendrán que ser bidimensionales.



Recursos

Módulo 1 y Módulo 2, temas 1-3, de los materiales de la asignatura.

Criterios de valoración

Cada uno de los ejercicios tiene indicada su puntuación en el propio enunciado.

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.

Se ha de entregar la solución en un fichero PDF. Adjuntad el fichero a un mensaje en el apartado Entrega y Registro de EC (REC). El nombre del fichero ha de ser ApellidosNombre_IA_PEC1 con la extensión .pdf (PDF).

La fecha límite de entrega es el **22 de marzo de 2019 (inclusive).**

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.